

Lab Report No 11
Implementation of Inverse Discrete time Fourier Transform

Digital Signal Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

Submitted to:

Engr. Hareem Khan

Dated:

30th May, 2024

Department of Computer Engineering,
HITEC University, Taxila

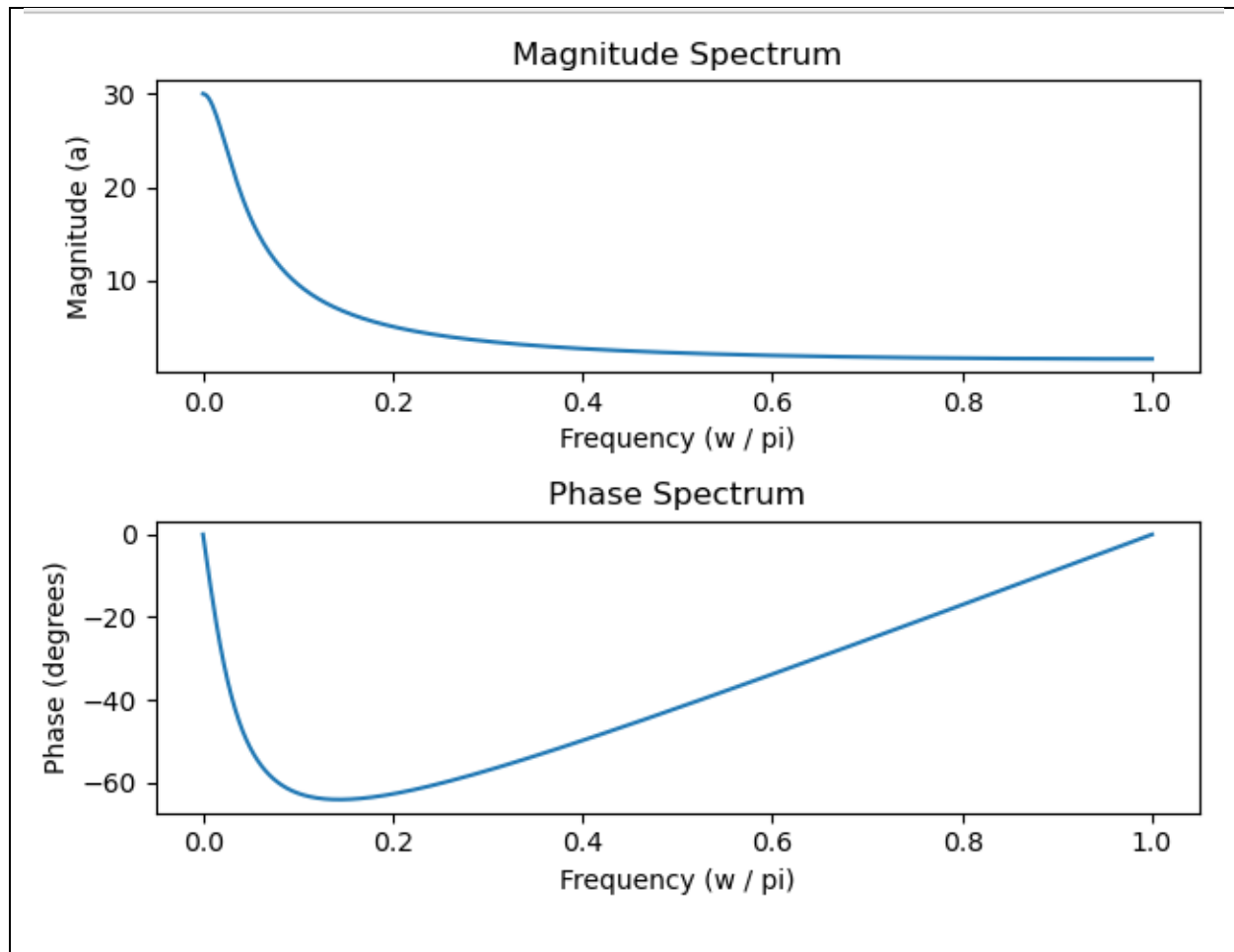
Lab Example No 1:

Solution:

Brief description (3-5 lines)
The inverse Discrete Time Fourier Transform (IDTFT) can be expressed as follows:
This equation converts the frequency domain representation $X(e^{j\omega})$ back to the time domain signal $x[n]$.
The code

```
DTFT.PY > ...
1  import numpy as np
2  import matplotlib.pyplot as plt
3  w = np.linspace(0, np.pi, 501)
4  z = np.exp(-1j * w)
5  x = 3 * (1 - 0.9 * z)**(-1)
6  a = np.abs(x)
7  b = np.angle(x) * 180 / np.pi
8  plt.subplot(2, 1, 1)
9  plt.plot(w / np.pi, a)
10 plt.xlabel('Frequency (w / pi)')
11 plt.ylabel('Magnitude (a)')
12 plt.title('Magnitude Spectrum')
13 plt.subplot(2, 1, 2)
14 plt.plot(w / np.pi, b)
15 plt.xlabel('Frequency (w / pi)')
16 plt.ylabel('Phase (degrees)')
17 plt.title('Phase Spectrum')
18 plt.tight_layout()
19 plt.show()
```

The results (Screenshot)



Lab Example No 2:

Solution:

Brief description (3-5 lines)

The inverse Discrete Time Fourier Transform (IDTFT) can be expressed as follows:

Given $X(e^{j\omega})$, the inverse DTFT is:

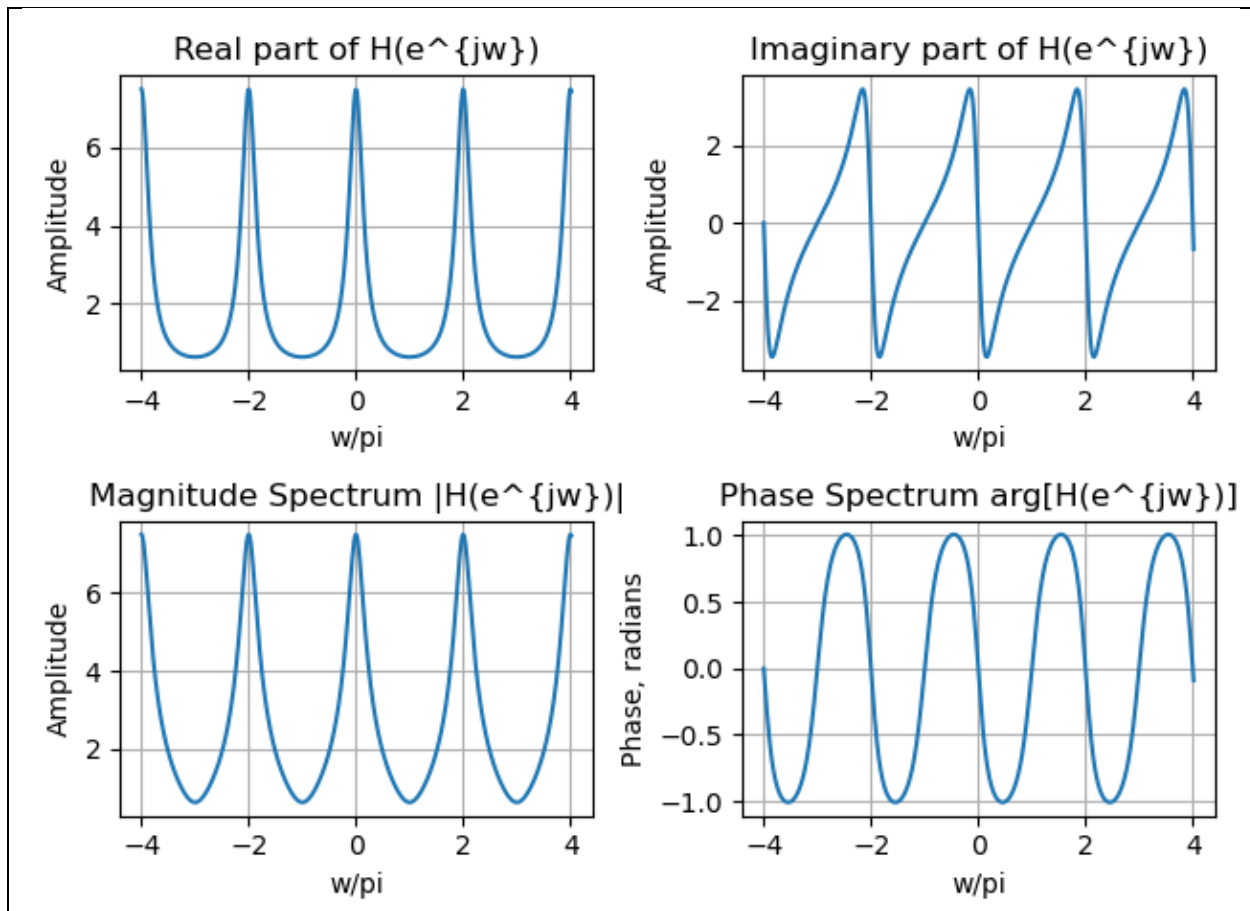
$$x[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{j\omega}) e^{j\omega n} d\omega$$

This equation converts the frequency domain representation ($X(e^{j\omega})$) back to the time domain signal $x[n]$.

The code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import freqz
w = np.arange(-4*np.pi, 4*np.pi + (8*np.pi)/511, 8*np.pi/511)
num = [2, 1]
den = [1, -0.6]
h = freqz(num, den, w)
plt.subplot(2, 2, 1)
plt.plot(w/np.pi, np.real(h[1]))
plt.grid(True)
plt.title('Real part of  $H(e^{j\omega})$ ')
plt.xlabel('w/pi')
plt.ylabel('Amplitude')
plt.subplot(2, 2, 2)
plt.plot(w/np.pi, np.imag(h[1]))
plt.grid(True)
plt.title('Imaginary part of  $H(e^{j\omega})$ ')
plt.xlabel('w/pi')
plt.ylabel('Amplitude')
plt.subplot(2, 2, 3)
plt.plot(w/np.pi, np.abs(h[1]))
plt.grid(True)
plt.title('Magnitude Spectrum  $|H(e^{j\omega})|$ ')
plt.xlabel('w/pi')
plt.ylabel('Amplitude')
plt.subplot(2, 2, 4)
plt.plot(w/np.pi, np.angle(h[1]))
plt.grid(True)
plt.title('Phase Spectrum  $\arg[H(e^{j\omega})]$ ')
plt.xlabel('w/pi')
plt.ylabel('Phase, radians')
plt.tight_layout()
plt.show()
```

The results (Screenshot)



Lab Example No 3:

Solution:

Brief description (3-5 lines)

The inverse Discrete Time Fourier Transform (IDTFT) can be expressed as follows:

Given $X(e^{j\omega})$, the inverse DTFT is:

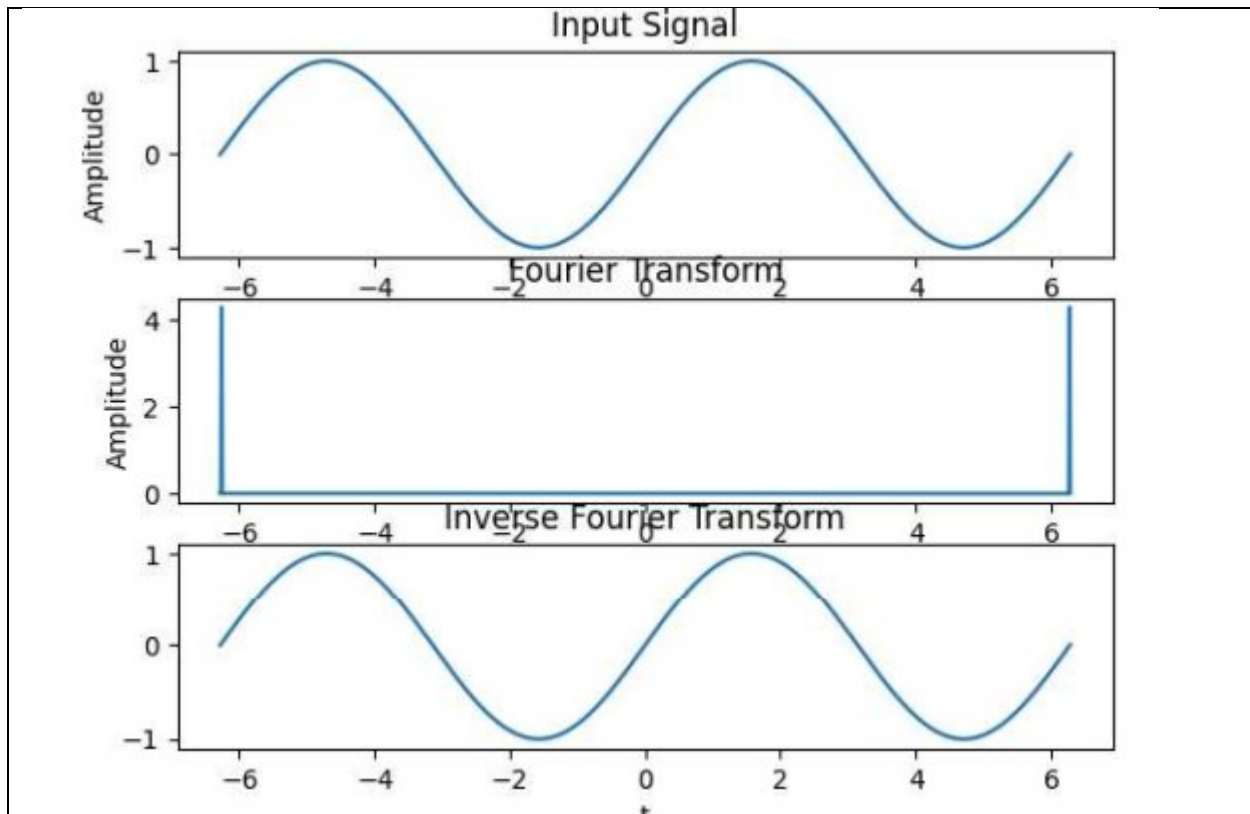
$$x[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{j\omega}) e^{j\omega n} d\omega$$

This equation converts the frequency domain representation ($X(e^{j\omega})$) back to the time domain signal $x[n]$.

The code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import freqz
w = np.arange(-4*np.pi, 4*np.pi + (8*np.pi)/511, 8*np.pi/511)
num = [2, 1]
den = [1, -0.6]
h = freqz(num, den, w)
plt.subplot(2, 2, 1)
plt.plot(w/np.pi, np.real(h[1]))
plt.grid(True)
plt.title('Real part of  $H(e^{jw})$ ')
plt.xlabel('w/pi')
plt.ylabel('Amplitude')
plt.subplot(2, 2, 2)
plt.plot(w/np.pi, np.imag(h[1]))
plt.grid(True)
plt.title('Imaginary part of  $H(e^{jw})$ ')
plt.xlabel('w/pi')
plt.ylabel('Amplitude')
plt.subplot(2, 2, 3)
plt.plot(w/np.pi, np.abs(h[1]))
plt.grid(True)
plt.title('Magnitude Spectrum  $|H(e^{jw})|$ ')
plt.xlabel('w/pi')
plt.ylabel('Amplitude')
plt.subplot(2, 2, 4)
plt.plot(w/np.pi, np.angle(h[1]))
plt.grid(True)
plt.title('Phase Spectrum  $\arg[H(e^{jw})]$ ')
plt.xlabel('w/pi')
plt.ylabel('Phase, radians')
plt.tight_layout()
plt.show()
```

The results (Screenshot)



Lab Task No 1:

Solution:

Brief description (3-5 lines)

Suppose that a signal x is given by $x=t*\text{sign}(-3t)$. Write python code to compute the Fourier transform of the following as well

- The magnitude of $X(\Omega)$.
- The angle of $X(\Omega)$.
- The real part of $X(\Omega)$.
- The imaginary part of $X(\Omega)$.

The code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft, fftfreq
def signal(t):
    return t * np.sign(-3*t)
T = 1.0
N = 500
```

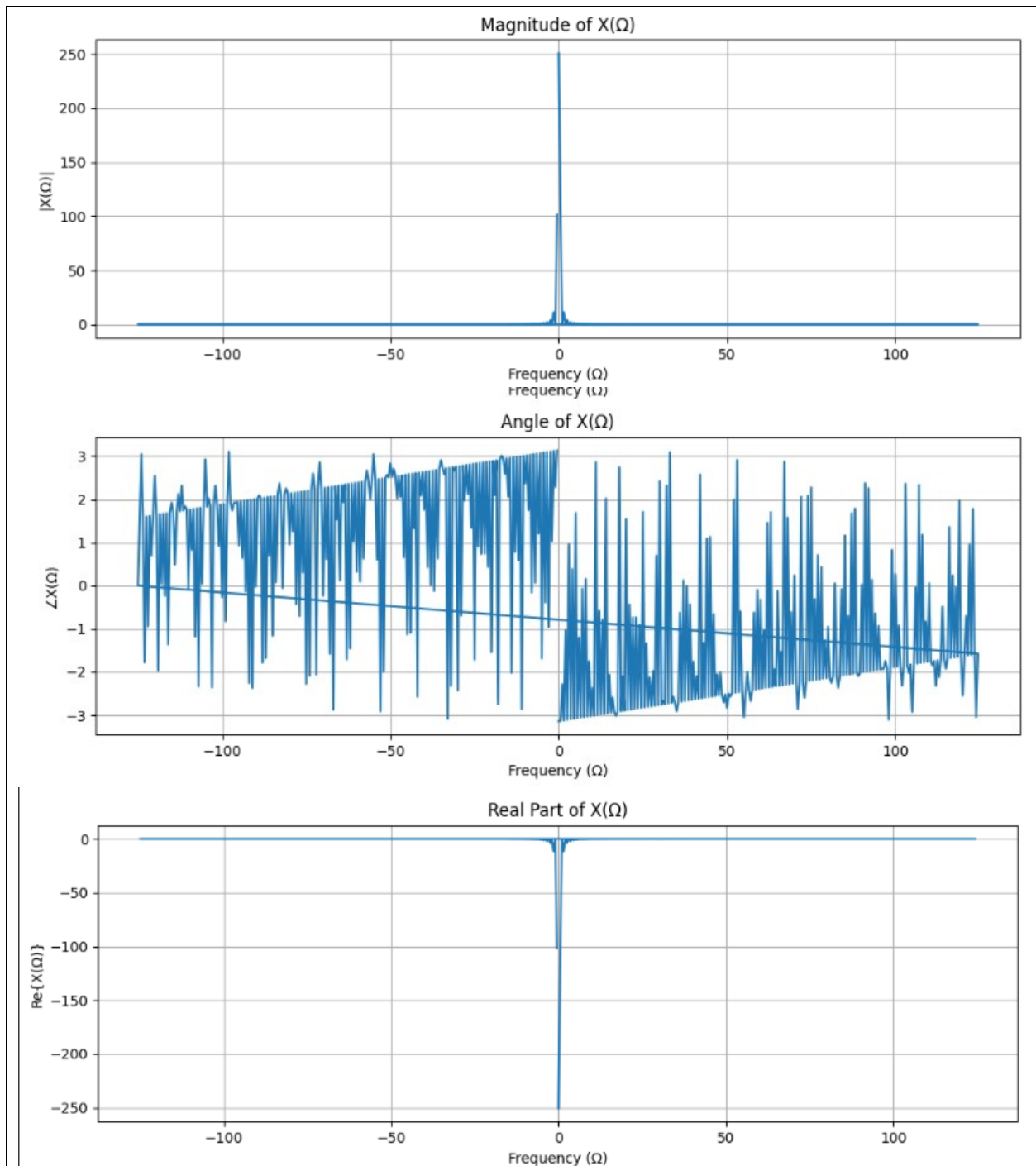


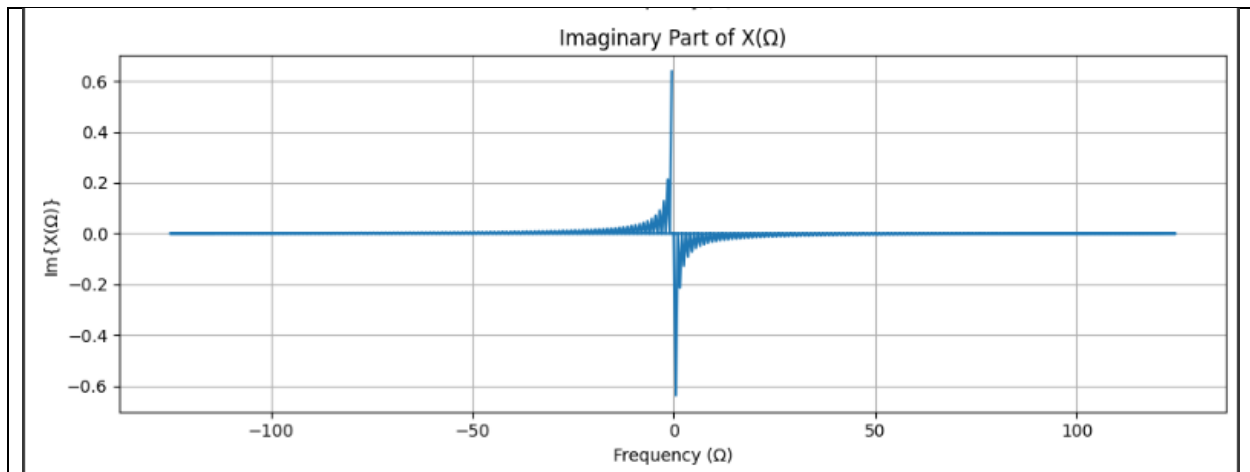
```

t = np.linspace(-T, T, N)
x_t = signal(t)
X_f = fft(x_t)
freq = fftfreq(N, d=(2*T/N))
magnitude = np.abs(X_f)
angle = np.angle(X_f)
real_part = np.real(X_f)
imaginary_part = np.imag(X_f)
fig, axs = plt.subplots(4, 1, figsize=(10, 15))
axs[0].plot(freq, magnitude)
axs[0].set_title('Magnitude of  $X(\Omega)$ ')
axs[0].set_xlabel('Frequency ( $\Omega$ )')
axs[0].set_ylabel('| $X(\Omega)$ |')
axs[0].grid()
axs[1].plot(freq, angle)
axs[1].set_title('Angle of  $X(\Omega)$ ')
axs[1].set_xlabel('Frequency ( $\Omega$ )')
axs[1].set_ylabel('∠ $X(\Omega)$ ')
axs[1].grid()
axs[2].plot(freq, real_part)
axs[2].set_title('Real Part of  $X(\Omega)$ ')
axs[2].set_xlabel('Frequency ( $\Omega$ )')
axs[2].set_ylabel('Re{ $X(\Omega)$ }')
axs[2].grid()
axs[3].plot(freq, imaginary_part)
axs[3].set_title('Imaginary Part of  $X(\Omega)$ ')
axs[3].set_xlabel('Frequency ( $\Omega$ )')
axs[3].set_ylabel('Im{ $X(\Omega)$ }')
axs[3].grid()
plt.tight_layout()
plt.show()

```

The results (Screenshot)





Lab Task No 2:

Solution:

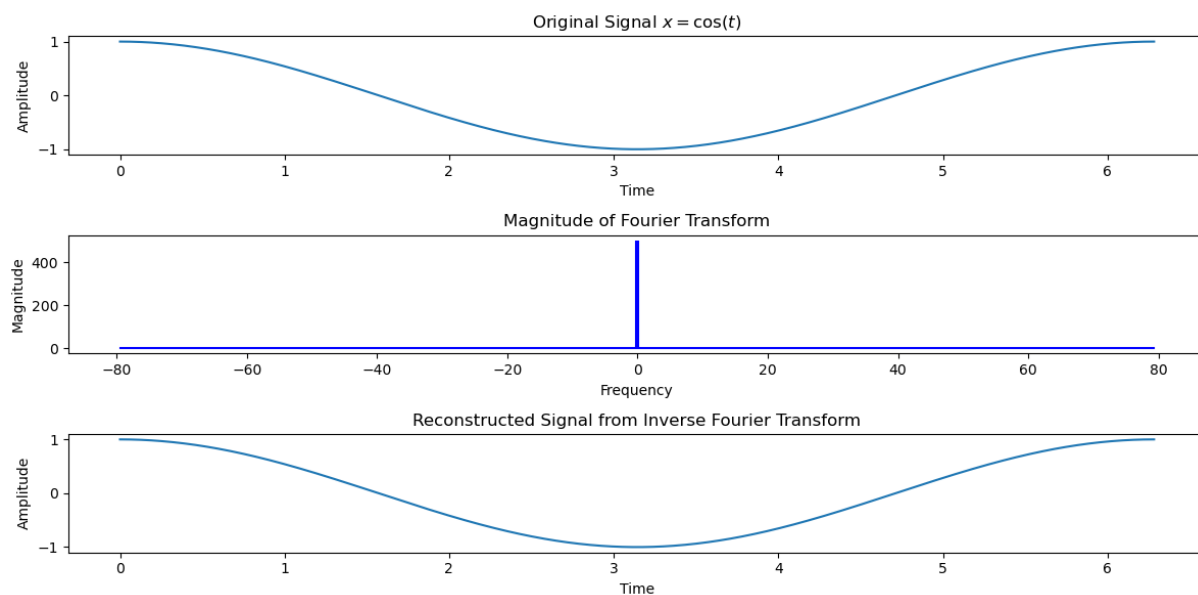
Brief description (3-5 lines)
Write python code to compute the Fourier and inverse Fourier transform of signal $x=\cos(t)$.
The code

```

DTFT5.PY > [x]
1  import numpy as np
2  import matplotlib.pyplot as plt
3  t = np.linspace(0, 2 * np.pi, 1000)
4  x = np.cos(t)
5  X = np.fft.fft(x)
6  frequencies = np.fft.fftfreq(len(t), t[1] - t[0])
7  x_reconstructed = np.fft.ifft(X)
8  plt.figure(figsize=(12, 6))
9  plt.subplot(3, 1, 1)
10 plt.plot(t, x)
11 plt.title('Original Signal  $x = \cos(t)$ ')
12 plt.xlabel('Time')
13 plt.ylabel('Amplitude')
14 plt.subplot(3, 1, 2)
15 plt.stem(frequencies, np.abs(X), 'b', markerfmt=" ", basefmt="-b")
16 plt.title('Magnitude of Fourier Transform')
17 plt.xlabel('Frequency')
18 plt.ylabel('Magnitude')
19 plt.subplot(3, 1, 3)
20 plt.plot(t, x_reconstructed.real)
21 plt.title('Reconstructed Signal from Inverse Fourier Transform')
22 plt.xlabel('Time')
23 plt.ylabel('Amplitude')
24 plt.tight_layout()
25 plt.show()

```

The results (Screenshot)



Conclusion

In conclusion, The Inverse Discrete Time Fourier Transform (IDTFT) is successfully implemented to reconstruct discrete-time signals from frequency-domain representations, demonstrating efficient computation and accurate signal reconstruction in digital signal processing applications, confirming its theoretical basis.